3. Build a Pod

- Basics of Linux primitives
- Using Linux primitives in Kubernetes
- Building a pod without Docker
- Why certain Kubernetes plugins have evolved over time

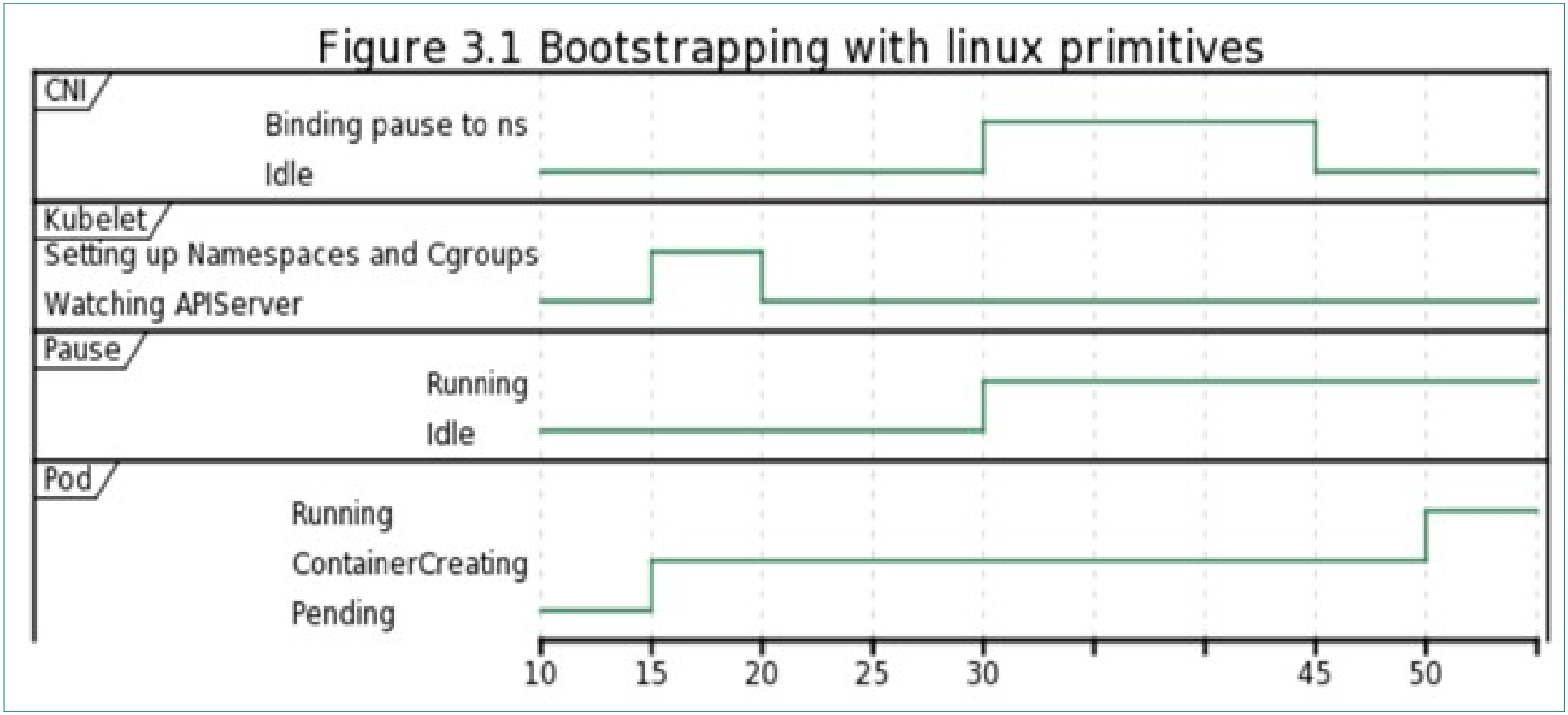
- Uses kind, see kind site to install and use
- Kind runs on top of docker which you already have
  - DockerFile to build kind

Basic Usage of kind

- After installation, have new command, kind
- `kind create cluster` creates Kubernetes
- `kind delete cluster` removes the cluster
- `kubectl cluster-info --context kind-kind` displays info on cluster
- Use `kubectl` (`oc` as well) to inspect nodes, pods, etc.
- `docker ps` will show container running `kind`
- `docker exec -t -i` *xxxxx* `/bin/sh` gets into node (then try `ps aux`) where *xxxxx* is container id

Steps to create a Pod in K8S

- Simplified timeline of creating a Pod



Pause Container

- Placeholder for pod to live in
  - first IP address
  - Linux namespaces
  - cgroups for resource limits
  - initial configuration (also called pod's sandbox)
- All containers in pod share same IP configuration
- Helps graceful termination of child processes

Linux Primitives

- Linux primitives are building blocks of Linux OS
  - commands we have seen, as well as iptables, DNS, and new things such as Linux namespaces and control groups
- Kube-proxy creates iptables rules
  - can see by running `iptables -L` in a Node
- Container Storage Interface (CSI) defines a socket for communication between kubelet and storage technologies
- `unshare` and `mount` used when creating containers

Prerequisites for Running Pod

- Need to implement isolation, networking and process management
- Useful Linux commands related to pods
  - `swapoff` – disables memory swapping
  - `iptables` – firewall and networking rules
  - `systemd` – starts kubelet as service
  - `socat` – establish bidirectional stream between processes
  - `nsenter` – enter into a namespace
  - `unshare` – isolate a process (from network, mount, PID)
  - `lsns` – list the namespaces for a process
  - `ps` – list running processes (check for health)

Running a Simple Pod

- Create a pod by running `kubectl create -f pod.yaml`
  - with yaml file on next slide
- Node that the kind field is set to "Pod" indication we are creating a Pod
- The busybox image is a minimal linux image we can use to check container behaviour
  - it has basic services, such as `ip`
- The webapp-port is not used
  - defines a loadbalanced endpoint could be behind service

pod.yaml

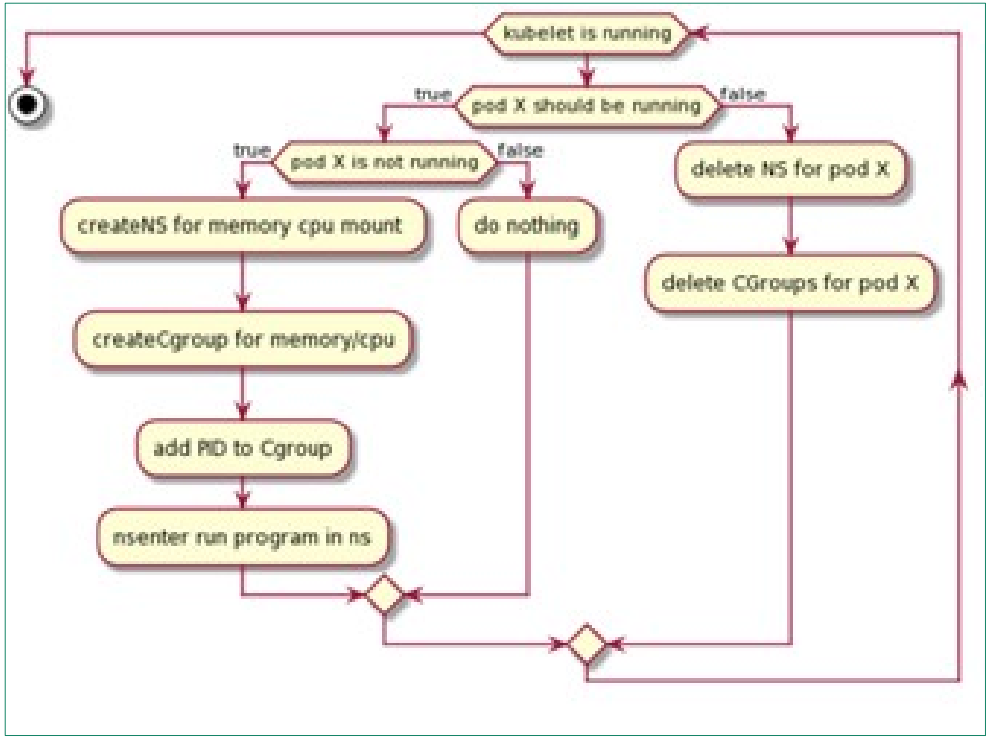
```yaml
apiVersion: v1
kind: Pod
metadata:
  name: core-k8s
  labels:
    role: just-an-example
    app: my-example-app
    organization: friends-of-manning
    creator: jay
spec:
  containers:
    - name: any-old-name-will-do
      image: docker.io/busybox:latest
      command: ['sleep','10000']
      ports:
        - name: webapp-port
          containerPort: 80
          protocol: TCP
```

Pod's Linux Dependencies

- Our pod needs access to CPU, memory, disk, networking, etc., as all programs do
- kubelet sets up the environment for the pod
  - creates isolated home for resources
  - adds a working network connection
  - gives Pod access to DNS
  - starts the Pod's programs
  - waits for the pod programs to exit and cleans up
- Can think of this as a Pod lifecycle

Pod Lifecycle

- Kubelet is continually checking
  - ∃ a pod that needs to start or be deleted?
- To start Pod, need to
  - create namespace
  - create control group
  - add Pod to these
- To stop Pod, need to
  - delete NS and CGroup of Pod



Checking Status of Pod

- Once a pod is running, a status section is added
  - can see with command: `kubectl get po -o yaml`
- Can output as JSON if you prefer (`-o json`)
- JSON path can be used to filter output:
  - `kubectl get pods -o=jsonpath='{.items[0].status.phase}'`
  - `kubectl get pods -o=jsonpath='{.items[0].status.podIP}'`
  - `kubectl get pods -o=jsonpath='{.items[0].status.hostIP}'`
- Using jsonpath helps learn the structure of the datamodels for objects in the cluster

Looking at Data Mounted in Pod

- Can run mount command inside pod using kubectl exec
  - e.g. `kubectl exec -t -i core-k8s -- mount | grep resolv.conf`
  - shows file resolve.conf is mounted from another location
  - these typically correspond to a folder under /var/lib/containerd on the node
- Poking around, you can see our pod has
  - a certificate to access the API server
  - an IP address on the 10 subnet different from the host IP
  - PIDs starting at 1 as if it were its own computer

Building a Pod from Scratch

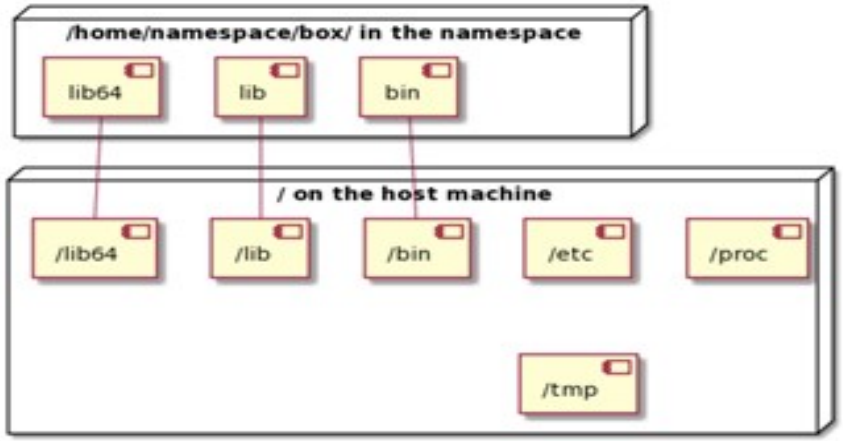
- Suppose we did not have Kubernetes and wanted a container management system
- Would need to handle (using Linux OS):
  - storage
  - IP addressing
  - network isolation
  - process identification
- Can start with basic Linux OS in our `kind` cluster
  - `docker ps | grep kind | cut -d' ' -f 1` gets container id
  - `docker exec -t -i container_id /bin/sh` enters

Install some tools

- Running in the Kind container we can update the OS and install some tools
  - (using apt-get as Kind is based on Ubuntu)
  - `apt-get update -y`
  - `apt-get install vim` (or `nano` or `emacs`)
- After the `docker exec` we are using a command prompt on the Ubuntu Linux running the container
  - but running as root, so be careful

Create an Isolated Process with `chroot`

- Create a folder which can only run a bash shell
- Three steps
  - decide what program to run
  - create environment (shared libraries required)
  - copy program into the chrooted location



Bash Script chroot.sh

- Save script to file chroot.sh using editor on Kind container
- in chroot "jail", normal commands not there
  - try cat
- Copied commands see limited file system
- ps can see outside processes
- exit or ctrl-D to leave

```
mkdir -p /home/namespace/box

mkdir /home/namespace/box/bin
mkdir /home/namespace/box/lib
mkdir /home/namespace/box/lib64
mkdir /home/namespace/box/proc

cp -v /usr/bin/kill /home/namespace/box/bin
cp -v /usr/bin/ps /home/namespace/box/bin
cp -v /bin/bash /home/namespace/box/bin
cp -v /bin/ls /home/namespace/box/bin
cp -v /bin/touch /home/namespace/box/bin
cp -v /bin/rm /home/namespace/box/bin

cp -r /lib/ /home/namespace/box/
cp -r /lib64/ /home/namespace/box/

mount -t proc proc /home/namespace/box/proc

chroot /home/namespace/box /bin/bash
```

Using Mount to Provide Data

- Containers need access to storage from somewhere else
  - as the container does not come with disks
- Can use `mount` command (with `--bind` option) to mount something from the underlying OS into our container
  - bind allows mounting an existing folder in another place

Securing Process with `unshare`

- If run ps inside chroot jail described above, you get real PIDs
  - even though you can only see the ones under the program running (bash)
- Unfortunately although you can only see "your" files, you can kill other processes if you know (or guess) their PIDs
  - you are not isolated enough
- The `unshare` command isolates the process space, network space

Eight Namespaces

- See namespaces man page
  - Cgroup isolates CGroup root directory (since Linux 4.6)
  - IPC isolates System V IPC, POSIX queues (3.0)
  - Network isolates network devices, stacks, ports (3.0)
  - Mount isolates mount points (3.8)
  - PID isolates process IDs (3.8)
  - Time isolates boot and monotonic clocks (5.6)
  - User isolates user and group IDs (3.8)
  - UTS isolates hostname and NIS domain name (3.0)

Namespaces: unshare

- unshare command
  - run program with some namespaces unshared from parent
  - see man pages (and more discussion below)

```
[root@bastion ~]# unshare --fork --pid --mount-proc
[root@bastion ~]# ps
    PID TTY          TIME CMD
      1 pts/0    00:00:00 bash
     23 pts/0    00:00:00 ps
[root@bastion ~]# logout
[root@bastion ~]# ps
    PID TTY          TIME CMD
 666027 pts/0    00:00:00 bash
 666081 pts/0    00:00:00 ps
[root@bastion ~]#
```

look in /proc for process information

Only 2 processes!

`unshare` Command Basics

- unshare [options] program [arguments]
  - unshares the indicated (via options) namespaces from the parent process and executes specified program
  - -i unshare IPC namespace
  - -m unshare mount namespace
  - -n unshare network namespace
  - -p unshare PID namespace
  - -u unshare UTS namespace
  - -U unshare user namespace
  - -C unshare cgroup namespace
  - -T unshare time namespace
  - -f fork the program as a child process of unshare

unshare Examples

- With above chroot setup
  - `unshare -p -f --mount-proc=/home/namespace/box/proc chroot /home/namespace/box /bin/bash`
    - isolates PID namespace, mounts proc folder, and forks to run chroot command
  - `unshare -p -n -f --mount-proc=/home/namespace/box/proc chroot /home/namespace/box /bin/bash`
    - also isolates network (so no network until we set it up)

Namespaces: nsenter

- nsenter command - join a given namespace

```
Usage:
 nsenter [options] [<program> [<argument>...]]

Run a program with namespaces of other processes.
Options:
 -a, --all              enter all namespaces
 -t, --target <pid>     target process to get namespaces from
 -m, --mount[=<file>]   enter mount namespace
 -u, --uts[=<file>]     enter UTS namespace (hostname etc)
 -i, --ipc[=<file>]     enter System V IPC namespace
 -n, --net[=<file>]     enter network namespace
 -p, --pid[=<file>]     enter pid namespace
 -C, --cgroup[=<file>]  enter cgroup namespace
 -T, --time[=<file>]    enter time namespace
 -U, --user[=<file>]    enter user namespace
 -S, --setuid <uid>     set uid in entered namespace
 -G, --setgid <gid>     set gid in entered namespace
```

Adjusting CPU Usage with Cgroups

- Cgroups allow us to give fixed amounts of CPU and memory to containers or pods
  - these can be set in policy or yaml files.
- Can set limits on max, min, default resources an object can use, as well as requests that it would like
- CPU in units of millicores, cores
  - e.g. 100m, 10
- Memory in Mi, Gi
  - e.g. 4Mi, 2Gi

greedy-pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: core-k8s-greedy
spec:
  containers:
    - name: any-old-name-will-do
      image: docker.io/busybox:latest
      command: ['sleep','10000']
      resources:
        limits:
          memory: "200Mi"
        requests:
          memory: "100Mi"
```

Managing Resources by Hand

- Going through process similar to Kubernetes, start with process (as earlier chroot example)
- Get PID of process, substitute for PID in box:

```
# mkdir /sys/fs/cgroup/memory/chroot0
# echo "10" > /sys/fs/cgroup/memory/chroot0/memory.limit_in_bytes
# echo "0" > /sys/fs/cgroup/memory/chroot0/memory.swappiness
# echo PID > /sys/fs/cgroup/memory/chroot0/tasks
```

- Go back into chroot box and any command will fail due to not enough memory!
- Of course we would give the process enough memory to do its job but not enough to interfere with others

Going into the World: Networking

- Typical microservice needs to communicate with other services
  - requires an IP, but also DNS to find the other services
  - may need certificates or credentials as well
- When processes are normally created, they share the host IP
  - need to be able to give our proto-container its own networking

The Networking Problem

- A container may need to have network traffic routed to it, from it, access DNS, or receive traffic from a loadbalancer
- Pods have text labels to identify and characterize them
- Information about Pod IPs is communicated to API server
- kube-proxy is responsible for forwarding traffic to pods
- Here, we discuss Linux primitives involved

Using `iptables` to Process Network Traffic

- Recall iptables is a rule based tool that interacts with the kernel Netfilter
  - industry transitioning to nftables, see How to use nftables and nftables HOWTO
  - nftables also is rule based, uses *mostly* the same Netfilter infrastructure with iptables compatibility layer
- iptables can do more than just be a firewall (as we briefly discussed with NAT)
- ...

Using kube-dns Pod

- Good pod to study
  - runs in any kubernetes cluster
  - has no special privileges
  - uses pod network, not host network
  - already exists in your Kind cluster
- Investigate with `ip route`

•traffic going to veth devices
•made by network plugin
•look at iptables rules to see how veth used (next)
•`iptables-save | grep 10.244.0`

```
# ip route
default via 172.18.0.1 dev eth0
10.244.0.2 dev vethfc5287fa scope host
10.244.0.3 dev veth31aba882 scope host
10.244.0.4 dev veth1e578a9a scope host
172.18.0.0/16 dev eth0 proto kernel scope link src 172.18.0.2
```

Routing into DNS pod with `iptables` Rules

- If something is accessing KUBE-SVC-ERIFX rule send it to KUBE-SEP-IT2Z rule

```
-A KUBE-SVC-ERIFXISQEP7F7OF4 -m comment --comment "kube-system/kube-dns:dns-tcp" -m
statistic --mode random --probability 0.50000000000 -j KUBE-SEP-IT2ZTR26TO4XFPTO
-A KUBE-SVC-ERIFXISQEP7F7OF4 -m comment --comment "kube-system/kube-dns:dns-tcp" -j KUBE-
SEP-ZXMNUKOKXUTL2MK2
-A KUBE-SVC-JD5MR3NA4I4DYORP -m comment --comment "kube-system/kube-dns:metrics" -m
statistic --mode random --probability 0.50000000000 -j KUBE-SEP-N4G2XR5TDX7PQE7P
-A KUBE-SVC-JD5MR3NA4I4DYORP -m comment --comment "kube-system/kube-dns:metrics" -j KUBE-
SEP-ZP3FB6NMPNCO4VBJ
```

- then send to 10.244.0.2:53 (for DNS)

```
-A KUBE-SEP-6E7XQMQ4RAYOWTTM -s 10.244.0.3/32 -m comment --comment "kube-
system/kube-dns:dns" -j KUBE-MARK-MASQ
-A KUBE-SEP-6E7XQMQ4RAYOWTTM -p udp -m comment --comment "kube-system/kube-
dns:dns" -m udp -j DNAT --to-destination 10.244.0.3:53
-A KUBE-SEP-IT2ZTR26TO4XFPTO -s 10.244.0.2/32 -m comment --comment "kube-
system/kube-dns:dns-tcp" -j KUBE-MARK-MASQ
-A KUBE-SEP-IT2ZTR26TO4XFPTO -p tcp -m comment --comment "kube-system/kube-
dns:dns-tcp" -m tcp -j DNAT --to-destination 10.244.0.2:53
```

Other Issues

- Storage
  - managed by Kubernetes StorageClasses PersistentVolumes and PersistentVolume Claims
- Scheduling
  - Pods scheduled based on resources is is asking for and available resources
  - As well as affinity, antiaffinity, etc. as mentioned above
- Upgrades and restarts
  - restarting a pod or adding new pods to a service (scaling) means IPs are changing – need to adjust all routing

Section Summary

- Kubernetes uses a collection of normal Linux primitives
- You can build a pod-like construct using `chroot`
- Storage, scheduling and networking need to be managed carefully to run in a production scenario
- Kubernetes was designed to handle the large number of challenges associated with running containers in production